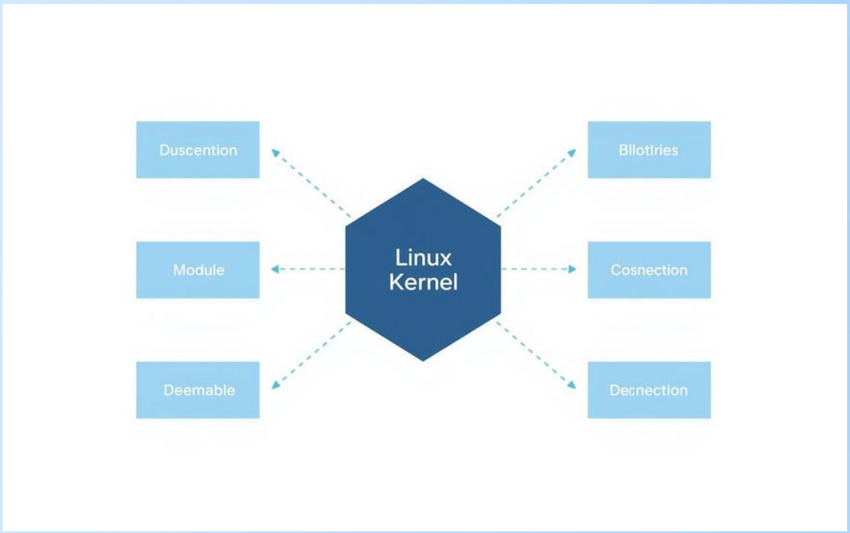
LINUX KERNEL MODULES & THE EUDYPTULA CHALLENGE FIRST

Understanding Kernel Modules
through a Hello World Example

Ankan Dey | Roll Number: 25AG10026

WHAT ARE LINUX KERNEL MODULES?

Kernel modules are loadable pieces of code that extend kernel functionality without rebooting. They can be loaded and unloaded on-demand, eliminating the need to recompile or reboot the kernel.
Use lsmod to list currently loaded modules on your system.



THE EUDYPTULA CHALLENGE FIRST TASK

Purpose: Introduce newcomers to Linux kernel development workflow.
Task: Write a kernel module that prints "Hello World!" on load and "Goodbye World!" on unload.
Requirement: Create a standalone Makefile to build the module for the current or specified kernel source.

KERNEL MODULE SOURCE CODE OVERVIEW

- Includes: linux/init.h, linux/module.h, linux/kernel.h
- Metadata macros define module information:
  - MODULE_LICENSE
  - MODULE_AUTHOR
  - MODULE_DESCRIPTION
  - MODULE_VERSION
- Two main functions structure the module:
  - hello_init: initialization function
  - hello_exit: cleanup function
- Registration macros link functions:
  - module_init(hello_init)
  - module_exit(hello_exit)

HELLO_INIT FUNCTION EXPLAINED

static int __init hello_init(void): Marks the function as initialization code that runs when the module loads. The __init macro tells the kernel this code can be discarded after initialization.

printk(KERN_DEBUG "Hello World!\n"): Prints a debug-level message to the kernel log buffer. printk is the kernel's equivalent of printf; KERN_DEBUG sets the log priority level. return 0 signals successful module loading.

HELLO_EXIT FUNCTION & MODULE REGISTRATION

static void __exit hello_exit(void): The __exit macro marks this function as cleanup code, called when the module is unloaded. It frees resources and prints "Goodbye World!" to the kernel log.

module_init(hello_init) and module_exit(hello_exit): These macros register the entry and cleanup functions. Proper cleanup is essential to prevent memory leaks and ensure system stability when unloading modules.

MAKEFILE PURPOSE & OVERVIEW

obj-m += linux_task.o: specifies the kernel module object file to build.
KDIR variable: path to kernel build directory (defaults to current running kernel source).
PWD variable: stores current working directory path.
all target: builds the module against kernel source.
clean target: removes build artifacts.

MAKEFILE DETAILED LINE-BY-LINE EXPLANATION

all target:
• Checks if KDIR directory exists using shell if statement
• Prints error message via echo and exits with exit 1 if missing
• Runs make -C $(KDIR) M=$(PWD) modules to build in kernel directory
• M=$(PWD) tells kernel build system where module source resides

clean target:
• Conditionally cleans based on KDIR presence
• If KDIR exists: runs kernel clean target
• If not: manually removes *.o, *.ko, *.mod files
• Removes temporary build folders (.tmp_versions, .cache.mk)
• Ensures clean state for fresh rebuilds

KEY TAKEAWAYS

Kernel modules allow dynamic extension of kernel functionality without rebooting the system. The Eudyptula Challenge first task introduces newcomers to the Linux kernel development workflow and module building process.

The hello.c module prints debug messages on load and unload via printk. The Makefile compiles the module correctly against the kernel source tree. Practice loading and unloading modules using insmod and rmmod commands.

THANK YOU FOR YOUR ATTENTION!

Ankan Dey | Roll No: 25AG10026